Pharmacy Manager

A desktop app for stock, expiry tracking and sales in a small pharmacy.
What it does, how it works, the tools used, and how it became a Windows .exe.

Platform	Windows 10/11 desktop app (single .exe)
Languages	HTML5, CSS3, JavaScript, Python 3.12
Core stack	Eel, PyInstaller (no database server needed)
Date	November 2026

Contents

1. Overview · 2. Using the app · 3. Architecture and data storage · 4. Tools and libraries · 5. Data model and business rules · 6. Front-end techniques · 7. Changes made for the desktop version · 8. Packaging as an .exe · 9. Problems, fixes and limitations · 10. Structure and next steps

1. Overview

Pharmacy Manager helps a small pharmacy keep track of its medicines, warns about stock that is running low or about to expire, and writes bills at the counter. It started as a single HTML page (HTML, CSS and JavaScript in one file) and was turned into a **desktop program**: the same page now runs in its own window, saves its data to a real file on the computer, and is packaged as one **PharmacyManager.exe** that needs no installation of Python.

Main features

- **Dashboard** with today's sales, number of medicines, items to reorder, expired or soon-to-expire batches, and the value of stock.
- **Inventory** with search, filters (all, low or out of stock, expired or expiring), and add, edit, restock and delete actions.
- **New sale** screen: search a medicine, build a bill, set a customer name and a percentage discount, complete the sale and print a receipt.
- **Sales history** listing every bill with totals, and the ability to reopen any receipt.
- Automatic **status tags** (Expired, Out of stock, Expires in N d, Low stock, OK) and safety checks that stop expired or out-of-stock items from being sold.
- Light and dark appearance (follows the Windows setting), works on small and large windows, and prints receipts cleanly.
- Amounts are shown in Bangladeshi Taka with thousands separators.

2. Using the app

Screen	What you can do
Dashboard	See sales today (and bill count), medicines listed, items that need reordering, items expired or expiring within 60 days, and total stock value at sale price. The *Reorder soon* list has a Restock button; the *Expiry watch* list shows batch number, date and units.
Inventory	Search by name, generic name or supplier; filter the list; **Add medicine** (brand, generic, form, supplier, batch, expiry, quantity, reorder level, sale price per unit). Each row has **Restock**, **Edit** and **Delete** (with a confirmation window).
New sale	Type to search by brand or generic name; click an item to add it to the bill. Change quantities (limited to what is in stock), remove lines, enter a customer name (default Walk-in) and a discount from 0 to 100 percent. **Complete sale** saves the bill, reduces the stock and opens the receipt.
Sales history	Shows the number of bills and total sales, and a table of every bill with date, customer, item count and total. **View** reopens the receipt, which can be printed again.

Demo data

On the very first start the app fills itself with seven sample medicines so every screen has something to show: Napa 500 mg, Seclo 20, Fexo 120, Maxpro 20, Zimax 500, Filmet 400 and Saline ORS. The dates are generated relative to today, so the samples always demonstrate the warnings: **Zimax 500** is already expired, **Fexo 120** expires in 35 days, and **Maxpro 20** (9 units, minimum 25) is low on stock. Edit or delete the samples and add the real stock before using the program in the shop.

3. Architecture and data storage

The program has two parts. The **page** (HTML/CSS/JavaScript) is everything you see and click. A very small **Python program** (`main.py`) does only one job: reading and writing the data file. They are joined by a library called **Eel**, which starts a local web server and opens the page in an Edge or Chrome app window with no address bar or tabs.

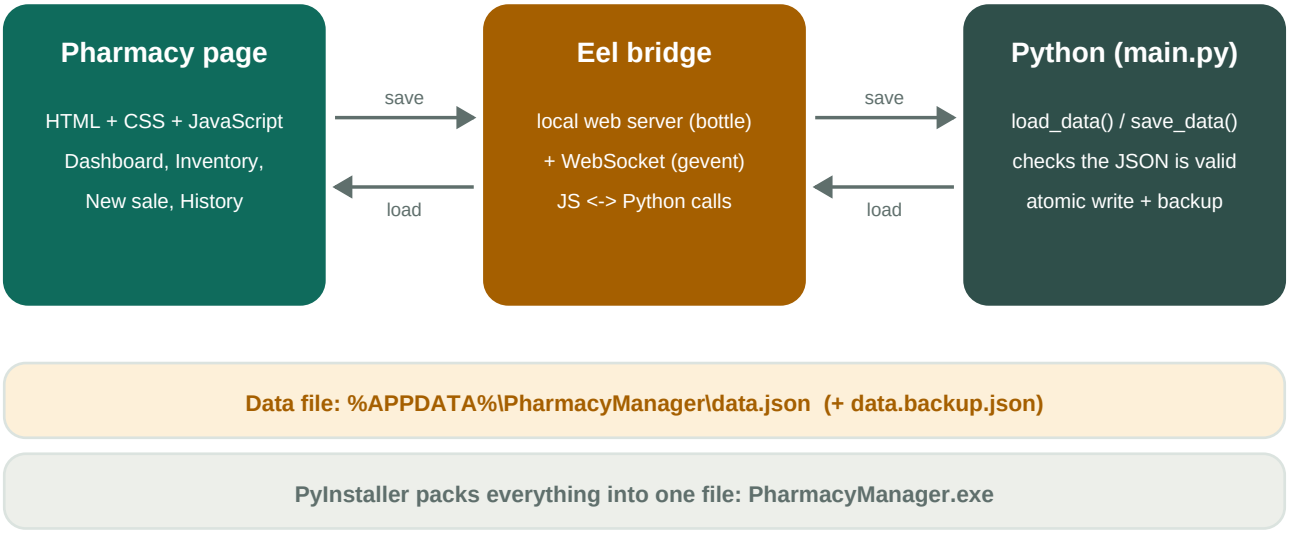

Figure 1 - The page, the Eel bridge, the Python file handling and the packaged exe.

How data is saved

- All data (medicines and sales) is one JSON document stored in `%APPDATA%\PharmacyManager\data.json`.
- **Safe writes:** Python first checks the text is valid JSON, writes it to a temporary file, copies the previous good file to `data.backup.json`, then swaps the new file in with one atomic operation. A power cut during saving cannot leave a half-written file.
- **Automatic recovery:** if the main file is ever damaged, the backup is loaded instead.
- **Saves are queued:** every change is sent to Python in order, so a fast sequence of clicks cannot save an older state over a newer one.
- **Fallbacks:** if the page is opened without Python (as a plain HTML file) it uses the browser's localStorage as before, and if the data file cannot be reached within 15 seconds at start-up it never overwrites real data with demo data.

Moving or backing up the data

To back up the pharmacy, copy the folder `%APPDATA%\PharmacyManager`. To move to another PC, install the exe there and copy the same folder across.

4. Tools and libraries

Versions are the ones installed in the project's virtual environment.

Tool / library	Version	What it is used for
HTML5, CSS3, JavaScript	ES2020	The whole application: screens, logic and styling in one file, with no framework and no external JavaScript libraries.

Tool / library	Version	What it is used for
Python	3.12	Runs the small backend and the build. A virtual environment (venv) keeps the packages separate.
Eel	0.18.2	Bridge between JavaScript and Python; serves the web folder and opens the app window.
bottle, gevent, gevent-websocket	0.13.4 / 26.9.0 / 0.10.1	Eel's dependencies: the local web server and the WebSocket connection.
PyInstaller	6.22.3	Builds the single PharmacyManager.exe (--onefile, --noconsole, --icon).
Pillow + NumPy	13.0.0 / 2.5.4	Used once, to draw the app icon from the logo's colors. They are **not** part of the running program.
Native browser features	-	<dialog> for pop-up windows, localStorage as a fallback, window.print() for receipts, CSS Grid, CSS variables, prefers-color-scheme.
Microsoft Edge / Chrome	installed on Windows	Displays the interface in app-window mode.
Google Fonts (Public Sans)	online only	The page's font. Offline it falls back to the system font (Segoe UI) with no loss of function.
VS Code, PowerShell, pip	-	Editing, running, installing packages and building the exe.

5. Data model and business rules

Everything lives in one object with two lists. Each sale keeps its own copy of the medicine name and price, so deleting a medicine later never changes old bills.

```
{
  "meds": [ { "id", "name", "generic", "form", "supplier",
              "batch", "exp" (YYYY-MM-DD), "qty", "min", "price" } ],
  "sales": [ { "id": "INV-0001", "t" (date-time), "cust",
               "items": [ { "name", "qty", "price" } ],
               "sub", "disc" (percent), "total" } ]
}
```

Rule	How it works
Status tag	Checked in this order: **Expired** (date before today) -> **Out of stock** (quantity 0) -> **Expires in N d** (60 days or less) -> **Low stock** (quantity at or below the reorder level) -> **OK**.
Reorder list	Medicines with quantity at or below the reorder level that are not expired.
Expiry watch	Medicines expiring within 60 days or already expired, sorted by date.
Stock value	Sum of quantity x sale price over all medicines (shown at sale price, not purchase cost).
Restock default	Suggested units = the larger of (2 x reorder level - current quantity) and 10. You can type any number above zero.
Adding to a bill	Only medicines that are in stock and not expired can be clicked. A line's quantity can never exceed the stock.
Completing a sale	Stock and expiry are checked again. Then each quantity is subtracted from stock, the bill is saved and the receipt opens.
Discount and total	Discount is limited to 0-100 percent. Total = round(subtotal x (100 - discount)) / 100, i.e. rounded to two decimals.

Rule	How it works
Bill numbers	INV- followed by the number of bills so far plus one, padded to four digits (INV-0001, INV-0002, ...).
Required fields	A medicine needs a brand name, an expiry date and a price. A negative quantity, reorder level or price is treated as zero.
Delete medicine	Asks for confirmation and removes the item from inventory only; past sales keep their records.

6. Front-end techniques

The page is plain JavaScript in a single file, about 18 KB. These are the main ideas.

- **One state object.** A global `S` holds `meds` and `sales`. `render()` draws the navigation and calls the function for the current tab (dash, inv, sell, hist).
- **Template strings.** Each screen builds its HTML as a string and puts it into the page; buttons call functions such as `restock(id)` directly.
- **Safe text.** A helper `e()` converts & < > and quotes into HTML entities before any typed name is shown, so special characters cannot break the page.
- **Native dialogs.** The add/edit form, restock window, delete confirmation and receipt all use the built-in `<dialog>` element with `showModal()`, giving a dimmed background and keyboard handling for free.
- **Toast messages.** A small fixed notice fades in and out for confirmations such as 'Medicine saved'.
- **Responsive layout.** CSS Grid with `auto-fit` columns for the statistic cards, two-column panels that become one column under 760 px, and tables that scroll sideways instead of breaking.
- **Theming.** Colors are CSS variables. The light theme is the default and the dark theme switches on automatically with the Windows setting.
- **Printing.** A print stylesheet hides the header, toast and buttons, so only the receipt appears on paper.
- **Dates.** Helper functions count whole days between today (local midnight) and an expiry date; this drives every warning.
- **Accessibility.** Visible keyboard focus outlines, labels on form fields, and screen-reader text on the Remove buttons and the toast.

7. Changes made for the desktop version

The page was supplied as a single HTML file. The desktop wrapper (the Python side, file storage, icon and packaging) was added with the help of an AI assistant (Claude). The look and every feature of the page were left as they were; only the storage code changed.

Area	Original browser page	Desktop version
Storage	localStorage inside the browser (lost if browser data is cleared)	data.json in %APPDATA%\PharmacyManager, with a backup copy and validation
Saving	Immediate, in the browser	Queued in order and written atomically by Python
Start-up	load() ran at once	load() waits for the file (up to 15 s), then the first screen is drawn
Old data	-	Data found in the browser's localStorage is copied into the file on first start

Area	Original browser page	Desktop version
Window	A browser tab	Its own 1200 x 800 window; the title bar shows the app name and icon
Icon	None	Custom icon (green and amber circle, like the page logo) for the window and the exe
Delivery	Open the HTML file	One PharmacyManager.exe; no Python needed on the target PC
Features and design	-	Unchanged

The storage change in the page is small: a queued `save()` that calls `eel.save_data(json)`, and an asynchronous `load()` that calls `eel.load_data()` before the first screen is drawn.

8. Packaging as an .exe

PyInstaller bundles the Python program, the packages and the web folder into one file. The command (type it on one line, from the project folder with the venv active):

```
pyinstaller --onefile --noconsole --clean --name PharmacyManager
            --icon icon.ico --add-data "web;web"
            --add-data ".venv\Lib\site-packages\eel\eel.js;eel" main.py
```

Option	Meaning
--onefile	Produce a single PharmacyManager.exe in the dist folder.
--noconsole	Do not show a black console window.
--clean	Clear PyInstaller's cache first, so old files and icons are not reused.
--icon icon.ico	Use the custom icon for the exe file.
--add-data "web;web"	Include the page (index.html and the favicon) inside the exe.
--add-data "...eel.js;eel"	Include Eel's JavaScript bridge file that the page loads.

Easiest way: double-click **build_exe.bat**. It creates the virtual environment, installs the packages, builds the exe and opens the dist folder. **run.bat** starts the app without building, which is handy for testing.

On the shop PC: copy dist\PharmacyManager.exe anywhere and double-click it. Edge or Chrome must be installed (Windows 10 and 11 include Edge). The first start takes a few seconds because a one-file exe unpacks itself.

The icon: the page's logo is a circle split into green and amber halves, tilted 35 degrees. The icon file reproduces it at seven sizes (16 to 256 px) and is used both for the exe (icon.ico) and for the app window (web/favicon.ico).

9. Problems, fixes and limitations

Problem	Cause	Fix
Data could disappear in a browser-only version	localStorage belongs to the browser profile and can be cleared.	Data now lives in a file in %APPDATA% with an automatic backup.
A power cut while saving could corrupt data	Writing straight into the data file.	Write to a temporary file, validate, back up, then replace atomically.
Older state saved over newer	Several saves could finish out of order.	Saves are chained so they run in the order they were made.
Demo data could overwrite real data	A slow or failed start-up read followed by a save.	If the file cannot be read, the app switches to the localStorage fallback and does not write demo data to the file.
python -m eel failed: No module named 'pkg_resources'	Newer setuptools no longer includes pkg_resources.	Call PyInstaller directly and add the web folder and eel.js with --add-data.
No module named 'eel'	Packages installed into a different Python than the one running the app.	Use run.bat / build_exe.bat, which always use the project's .venv.
Old icon keeps showing	Windows caches exe icons; old build/dist/.spec files and shortcuts keep the old one; the window icon is web/favicon.ico.	Delete build, dist and *.spec, replace both icon.ico and web/favicon.ico, rebuild with --clean, refresh Windows' icon cache and recreate shortcuts.
Antivirus warning on the exe	PyInstaller one-file programs are often flagged as false positives.	Add an exclusion, build with --onedir, or code-sign the exe.
Text looks different offline	Public Sans is loaded from Google Fonts.	Harmless fallback to Segoe UI. To keep the exact font, download it and load it from the web folder.

Notes and limitations

Fixed before publishing: 'Sales today' and expiry days now use local time. The original page compared bill times in UTC, so in Bangladesh (UTC+6) the 'Sales today' counter rolled over at 6:00 AM instead of midnight, and the sample expiry dates were one day early. Both now use the local date and give the same result in every time zone.

- Each medicine has a single batch and expiry date. A shop receiving several batches of the same medicine should add each batch as its own entry.
- One computer, one data file: there is no sharing between several PCs and no user login.
- A JSON file is perfectly fine for thousands of medicines and bills; for very large histories a database such as SQLite would be faster.

10. Structure and next steps

Project structure

```
PharmacyManager/
  main.py            Python: load_data() and save_data(), starts the window
  icon.ico           icon used for the .exe
  requirements.txt   eel, pyinstaller
  run.bat            create venv, install packages, start the app
  build_exe.bat      build dist\PharmacyManager.exe
  README.txt         short instructions
  web/
    index.html       the complete Pharmacy Manager page
    favicon.ico      window icon
```

Ideas for future work

- Export buttons for bills and stock (CSV or PDF) and a one-click backup and restore.
- Several batches per medicine, with the oldest expiry sold first.
- Purchase orders and a supplier list built from the reorder screen.
- Returns and refunds, VAT, and a thermal-printer receipt layout.
- Barcode scanning: most scanners type the code like a keyboard, so it can feed the search box.
- Daily sales report and an expiry alert shown when the program starts.
- User login with roles (owner and cashier) and an audit log of stock changes.